Homework — 2.1.3 Thinking Procedurally — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Part A (14 marks)

1. [3] Any three plausible sub-procedures (1 each): browse/select film & showing; check seat availability; let the user choose seats; take/validate payment; issue/email the ticket; update the seat-availability records; create the booking record. Must be plausible sub-tasks of the booking scenario.

2. [2] 1 mark: `issueTicket` depends on payment having succeeded. 1 mark: it uses the **output** of `takePayment` (a confirmed/successful payment); you cannot issue a valid ticket until payment is confirmed, so the order is fixed/sequential.

3. (a) [2] 1 mark for a valid fixed pair, e.g. "fill with water" must come before "heat water" (or "lock door" before "fill", or "wash" before "drain"). 1 mark: explain the dependency — e.g. you cannot heat water that has not yet been added; the later step needs the result/state produced by the earlier one.

(b) [1] Any reasonable answer with justification, e.g. starting a status-display update or a timer could run alongside the wash because it does not depend on the wash's result. *(Accept any sensible independent action; the justification is the mark.)*

4. [2] Any two (1 each): defined in a **single place**, so it can be called/reused for every question; a fix or change is made **once** and applies everywhere — avoiding duplicated, inconsistent code; easier to read/test/debug; shorter program.

5. [2] 1 mark: decomposition splits the game into independent sub-problems/modules (e.g. world rendering, combat, save system, AI). 1 mark: each module can be assigned to a **different programmer** to build and test in parallel, then integrated — making the large task manageable and faster to complete.

6. [2] 1 mark: sensible named, ordered sub-procedures, e.g. `receiveApplication` → `validateDetails` → `checkEligibility` → `takePayment` → `issuePermit`. 1 mark: identify a fixed-order pair with the dependency, e.g. `issuePermit` must follow `checkEligibility` because it uses its output (you cannot issue a permit until eligibility is confirmed). Must use named sub-procedures applied to the scenario.

Part B (6 marks)

7. [2] 1 mark: a precondition is a condition that must be **true before** a routine runs (correctly). 1 mark: for binary search, the list must be **sorted** into order beforehand.

8. [2] 1 mark + 1 mark, in order, any two of: **lexical analysis** (tokenising) → **syntax analysis** (parsing) → **semantic analysis** → (then code generation, with optimisation). Two correct stages that precede code generation, in the right order.

9. [2] 1 mark + 1 mark, any one developed advantage: Agile is **iterative** and welcomes changing requirements, delivering working software in increments **[1]**; so the customer sees progress early and

changes are cheaper to accommodate, whereas Waterfall fixes requirements up front and changes late are costly **[1]**.

Total: 14 + 6 = 20 marks